

SNAPDRAGON AI CHALLENGE SUBMISSION

StereoEdge-Grasp

Real-time 3D grasp synthesis from low-cost stereo vision, built for on-device inference on the Snapdragon NPU

Two ordinary cameras. No depth sensor. No cloud. A collision-checked 6-DoF grasp pose for a robot arm.

2

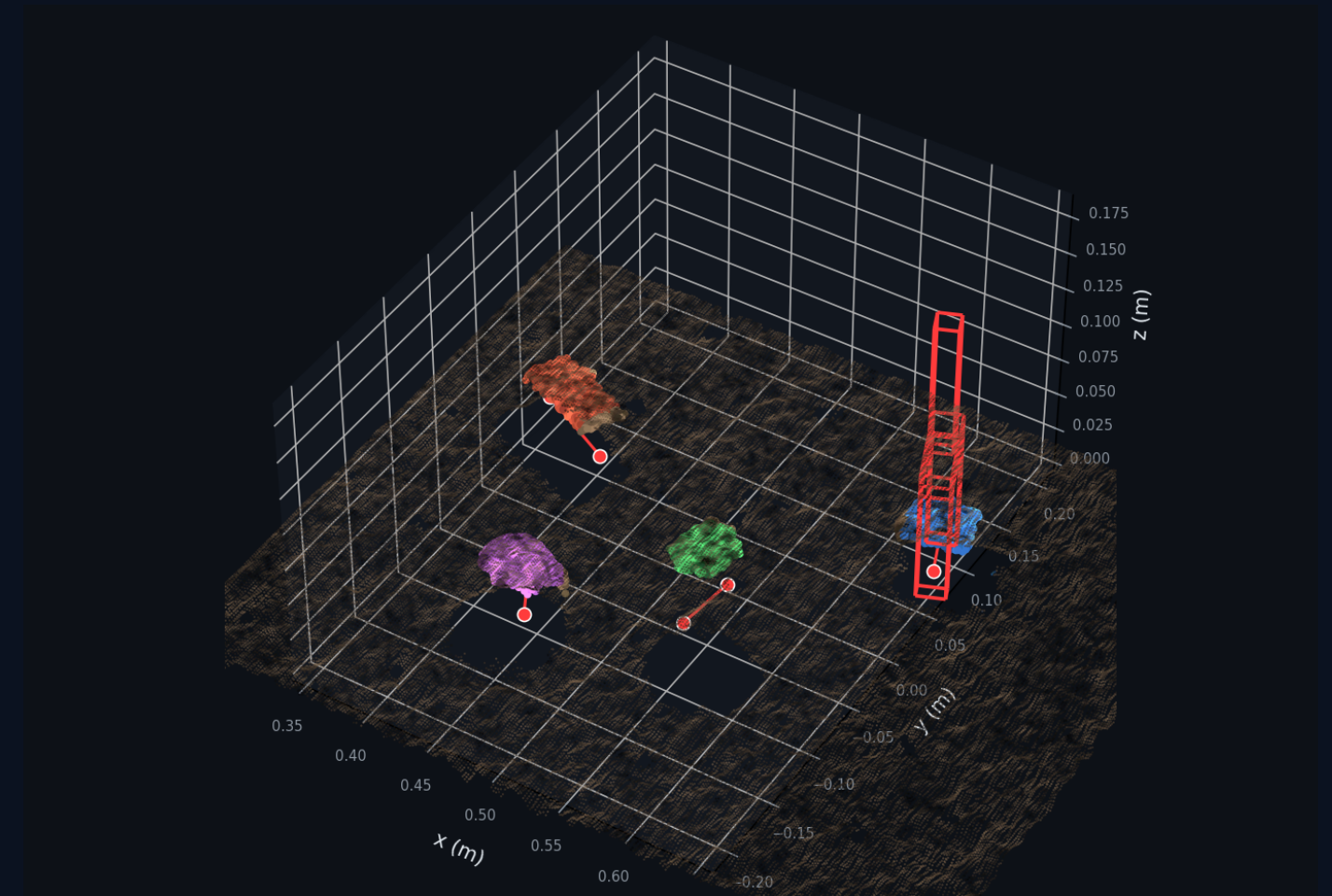
commodity cameras

0

bytes sent to the cloud

6-DoF

grasp + width + score



THE PROBLEM

Robot vision is too costly, too heavy, too exposed

Expensive sensors

RGB-D cameras add cost, bulk and cabling to every cell, and struggle in sunlight and on reflective parts.

Cloud round-trips

Streaming frames to remote GPUs adds latency and jitter to a control loop that must be deterministic.

Privacy and power

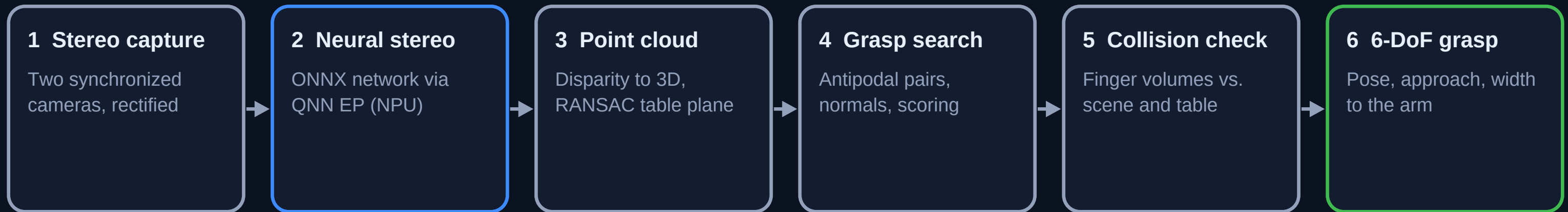
Factory and lab imagery is sensitive, and always-on cloud inference burns energy and bandwidth.

Our answer

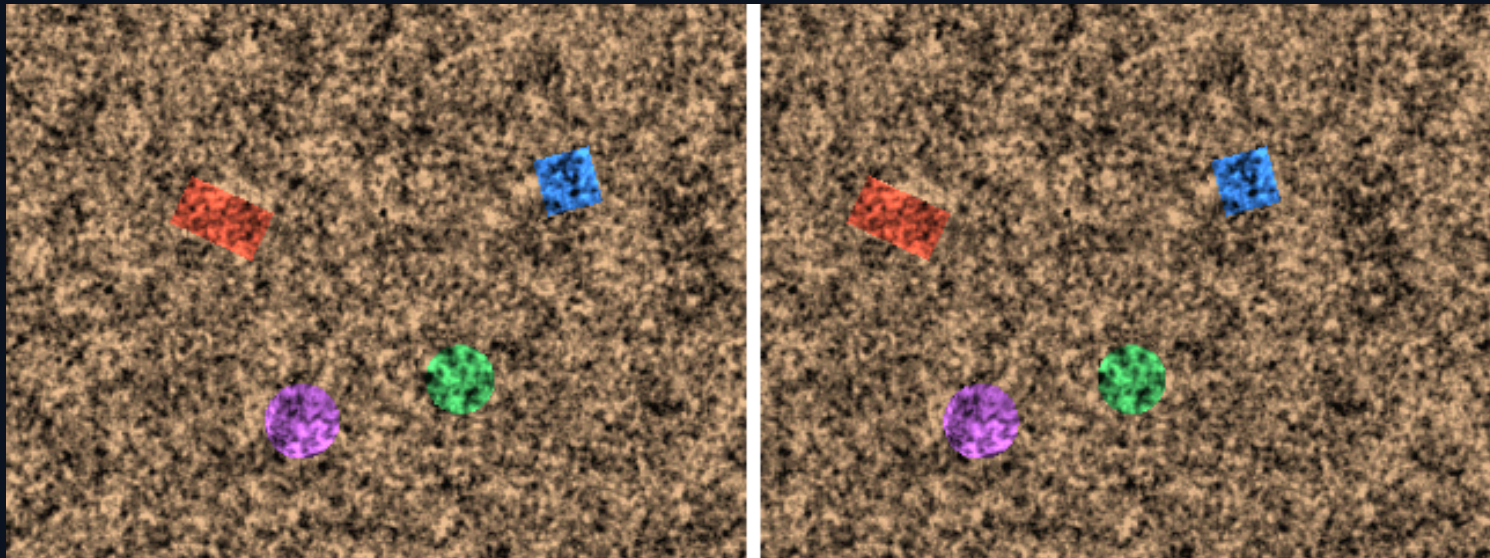
Recover 3D from two cheap cameras and compute the grasp locally: the matrix-heavy stereo network targets the Snapdragon NPU, and the whole loop stays on the device.

HOW IT WORKS

From two images to one grasp pose



Neural workload: target for the Hexagon NPU



Input: left / right frame (synthetic top-down scene with ground truth)

Swappable backends behind one interface

sgbm classical OpenCV baseline (CPU)

onnx-cpu ONNX graph on onnxruntime

onnx-qnn same graph on the Hexagon NPU (QNN HTP)

A trained stereo model from [Qualcomm AI Hub](#) or an open-source hub drops in with the same input/output contract.

Why it belongs on Snapdragon

Privacy by construction

The pipeline makes no network calls. Frames, point clouds and grasps stay on the HP Snapdragon PC.

Deterministic latency

No cloud round-trip. The stereo network is a static-shape ONNX graph of NPU-friendly ops (Pad, Slice, Sub, Abs, Concat, AveragePool).

Frees the CPU

Offloading the dense matrix math to the NPU leaves CPU cores for control, vision post-processing and the arm driver.

Low-cost scale-out

Commodity cameras replace depth sensors, and one PC serves as the cell's edge brain.

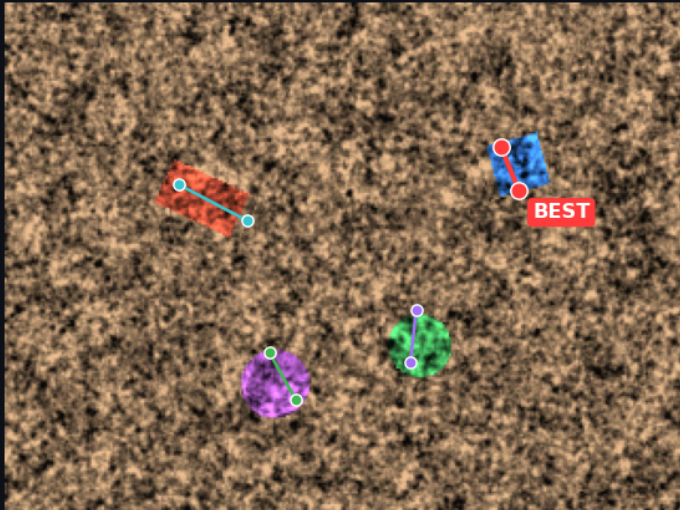
STATUS QNN backend, AI Hub profiling script and Windows-ARM64 setup are implemented. On-device NPU latency and power will be measured on Snapdragon X hardware and are not claimed here.

Live output: cloud, contacts and gripper pose

StereoEdge-Grasp | stereo pair → disparity → point cloud → 6-DoF grasp

backend: onnx-cpu

Left camera + grasp contacts



Disparity (px) invalid = dark

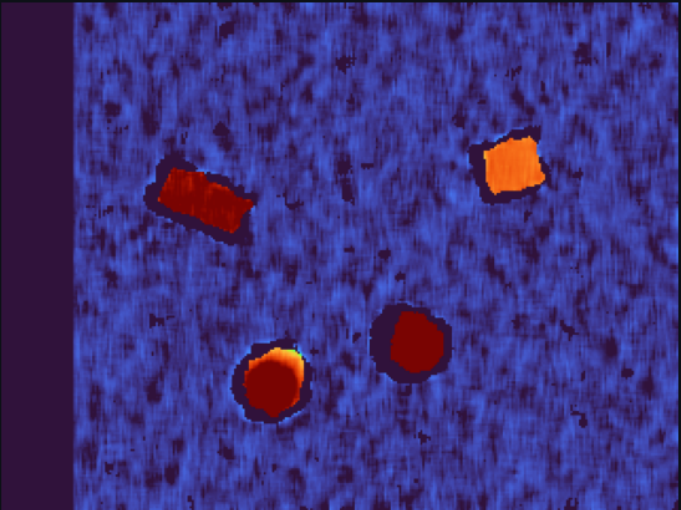
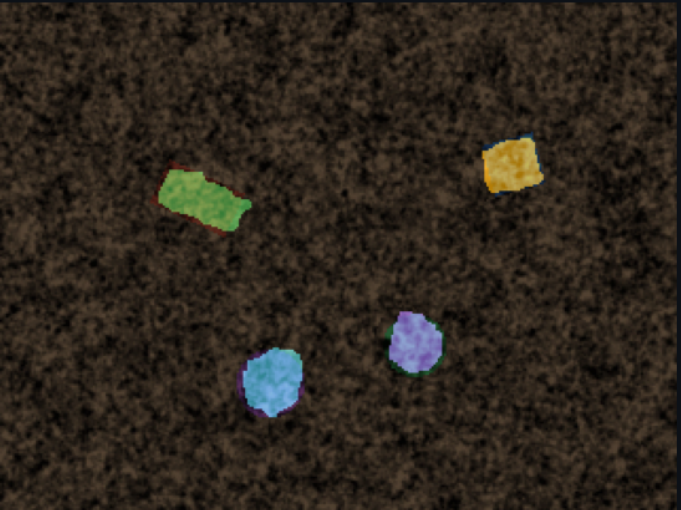
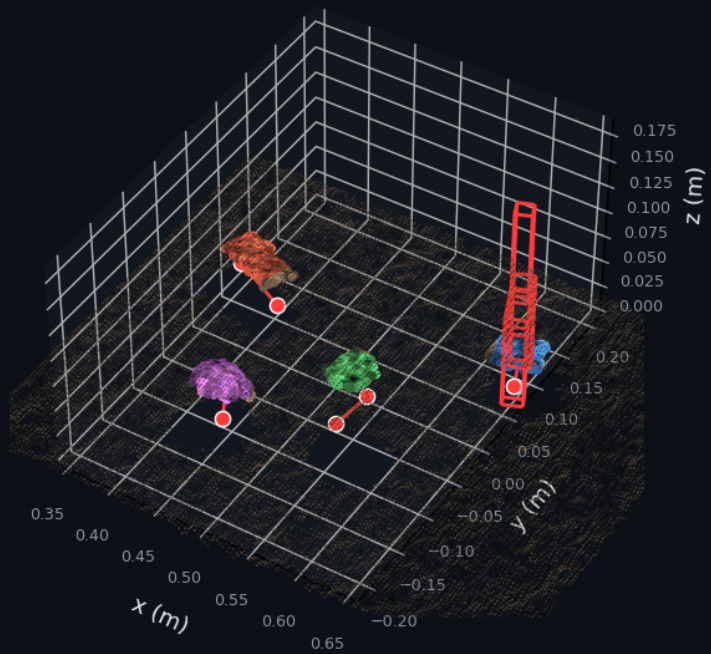


Table plane removed → 4 objects



3-D point cloud (robot-base frame) with gripper poses



LATENCY (this machine)

stereo_matching	22.4 ms
point_cloud	10.5 ms
plane_fit	12.7 ms
segmentation	2.1 ms
normals	12.7 ms
grasp_synthesis	15.4 ms
TOTAL	75.8 ms (13.2 FPS)

BEST GRASP (robot base frame)

position	[0.648, 0.079, 0.021] m
approach	[0.0, 0.0, -1.0]
closing	[0.38, -0.93, 0.0]
width	41.9 mm
score	0.964
antipodal	0.97
centering	0.98
width_fit	0.90
clearance	1.00
normal_agree	0.97

Red
selected grasp contacts
and gripper

Score
antipodal, centering,
width, clearance,
normals

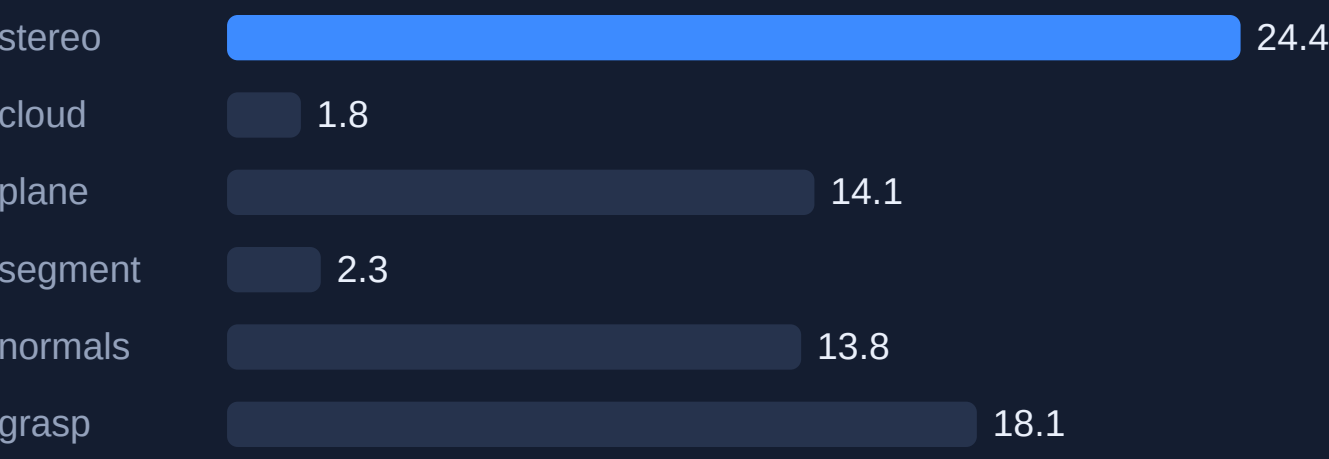
Timing
median of 5 runs, single
x86 CPU core

Measured results

Stereo accuracy vs. ground truth

	SGBM	ONNX net
Mean disparity error	0.27 px	0.08 px
Pixels within 1 px	99.4 %	100 %
Valid pixels	84.6 %	85.6 %
Objects found / grasped	4 / 4	4 / 4

Latency per stage (ms, median)



End-to-end: 75 ms = 13 FPS on one x86 CPU core (320 x 240, ONNX on CPU)

Setup: Linux x86-64, single core, no NPU, median of 30 runs on a synthetic scene with ground truth. The grasp poses land within 2 cm of true object centres with widths inside the gripper range (7 automated tests).

What this tells us

The stereo network is already the largest single stage, which is exactly the part the NPU is meant to take. The remaining stages are lightweight CPU code. We do not claim NPU speed-ups until measured on the device.

Where it applies

Bin picking & sorting

Mixed parts on a conveyor or tray

Small-batch manufacturing

Flexible cells without custom fixtures

Lab & pharma automation

Sensitive imagery must stay on site

Recycling & e-waste

Cheap sensing for harsh, dirty cells

Deployment

pip install, one Python codebase

HP Snapdragon PC as the cell's edge brain

Two USB/CSI cameras, standard calibration

JSON grasp output for any arm driver

MIT-licensed, runs on CPU without an NPU

Accessible by design: the same code runs on any machine for development and switches to the NPU with one flag (--backend onnx-qnn).

Honest limits and next steps

Known limitations

- Stereo needs texture; glossy or featureless parts degrade depth
- Demo scene is clean and top-down; clutter is harder
- Grasp search assumes vertical-walled objects
- NPU latency and power are pending hardware

Next steps

- Drop in a trained stereo model (AI Hub / open source), quantise to INT8
- Profile on Snapdragon X with AI Hub; publish NPU latency and power
- Validate on real stereo pairs
- Hardware-in-the-loop with a physical arm

Deliverables

Source repository (MIT) with README, 7 passing tests, benchmark script, Windows-ARM64 setup script, QNN backend and AI Hub profiling tool. Every figure in this deck is generated by the code in the repository.